Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 30.01.2024.

Ime i prezime: _____

Broj indeksa: _____

Broj bodova: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_PopravniFebruar_Test1_GrupaA_IT1g2022**
8. Ne postoji minimum bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**
10. **Da bi se test smatrao položenim, neophodno je osvojiti minimum 16 bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Pronalaženje elementa u redu (Queue Peek)*

Napisati metodu koja implementira standardan način za pronalaženje elementa sa vrha reda.

Dodatne napomene za rešavanje zadatka:

- Algoritam napisati u okviru zakomentarisane metode ***peek(int value).***
- Pomenuta metoda nalazi se u okviru klase ***Queue*** koja se nalazi u pridruženom template-u.
- Obezbediti kod u Main metodi za testiranje algoritma.

Zadatak 2 – *Jednakost suma ispod i iznad glavne dijagonale kvadratne matrice*

Napisati algoritam koji će proveriti da li se sume ispod i iznad glavne dijagonale kvadratne matrice poklapaju. Glavna dijagonala sadži sve elemente na putanji od polja koji se nalazi u gornjem levom ćošku, do polja koje se nalazi u donjem desnom ćošku (na narednoj slici nulama je označena glavna dijagonala).

$$\begin{bmatrix} 0 & 3 & 2 \\ 2 & 0 & 4 \\ 3 & 4 & 0 \end{bmatrix}$$

Dodatni zahtevi za rešavanje zadatka:

- Algoritam treba da rezultuje ispisom u konzoli u formatu: „Suma ispod i iznad glavne dijagonale se (ne) poklapa“.
- Algoritam pisati u okviru metode: ***public static void findSumBelowAndAboveMainDiagonal(long[,] matrix, int rows, int cols)***

Primer:

- Input: $\begin{bmatrix} 0 & 3 & 2 \\ 2 & 0 & 4 \\ 3 & 4 & 0 \end{bmatrix}, 3, 3$
- Output: „Suma ispod i iznad glavne dijagonale se poklapa“

- Input: $\begin{bmatrix} 0 & -1 & 2 \\ 2 & 0 & 4 \\ 3 & 4 & 0 \end{bmatrix}, 3, 3$
- Output: „Suma ispod i iznad glavne dijagonale se ne poklapa“

Zadatak 3 – *Kumulativna suma vrednosti levih suseda*

Napisati algoritam koji modifikuje listu tako što vrednost čvora zanemenjuje vrednošću sume svih elemenata koji se u listi nalaze sa njegove leve strane.

Dodatne napomene za rešavanje zadatka:

- Zahtevan potpis metode: ***public static void calculateCumulativeSum()***
- Metodu pisati u okviru klase ***LinkedList***

Primer:

- Input: ***LinkedList: 1 -> 0 -> 6 -> 8 -> 4***
- Output: ***0 -> 1 -> 1 -> 7 -> 15***

- Input: ***LinkedList: 1 -> 2 -> 4 -> 0 -> 6***
- Output: ***0 -> 1 -> 3 -> 7 -> 7***